HACKATHON STARTER KIT

Reinforcement Learning & AI Agents

New to reinforcement learning? This kit takes you from the core ideas to your first hands-on experiments with RL and LLM agents, with a curated set of free resources along the way.



Who it's for

Students and developers comfortable with Python, with no RL background needed

Time to work through

Roughly 12 to 18 hours, spread over a week or two before the event

What you need

A laptop, Python, Docker, and an API key from any LLM provider

01 Why RL and Agents Matter

Reinforcement learning (RL) is how machines learn from experience: by trying things, seeing what happens, and doing more of what works. It's behind some of AI's biggest milestones, and it's now central to how modern language models and AI agents are trained.

Today's AI agents don't just answer questions: they write code, browse websites, operate software and run multi-step workflows. Understanding RL, how agents act, and how their behaviour is measured gives you a foundation for almost any project in this space.

GAMES

Superhuman play

Systems like AlphaGo mastered Go and other games largely through self-play.

ROBOTICS

Learning to move

Robots learn to walk and grasp objects through trial and error in simulation.

LANGUAGE MODELS

Better assistants

RL from feedback and rewards helps LLMs follow instructions, reason and use tools.

What you'll learn in this kit

#	SECTION	YOU'LL COME AWAY ABLE TO…
02	RL in One Page	Explain the agent–environment loop and the core vocabulary
03	From Classic RL to LLM Agents	See how the same ideas apply when the agent is a language model
04	Hands-On: Your First RL Loop	Read and write a minimal RL setup in Python
05	Learning Path	Follow a four-stage plan from zero to your first experiments
06	Resource Library	Pick the right free courses, docs and libraries
07	Warm-up Projects	Practise with three small hands-on builds
08	Get Ready Checklist	Arrive with your setup working

02 RL in One Page

Reinforcement learning is learning by trial and error. An agent tries actions in an environment, gets rewarded for good outcomes, and gradually learns what works.



Core vocabulary

Agent
The decision-maker: a robot, a game-playing program, or an LLM

Environment
Everything the agent interacts with, including the rules of the world

State / Observation
The situation right now; the observation is the part the agent can see

Action
A choice the agent makes: a move, a button press, a tool call

Reward
A number telling the agent how good the outcome was

Episode
One full attempt from start (reset) to finish (termination)

Policy
The agent's strategy for picking actions given what it observes

Return
The total reward collected over an episode; what the agent tries to maximise

Exploration vs exploitation
Trying new things to learn, versus repeating what already works

Reward shaping
Giving partial credit for progress so the agent gets signal before full success

Sparse reward
Reward only at the very end, which makes learning slower and harder

Reward hacking
When an agent scores well in a way the designer never intended

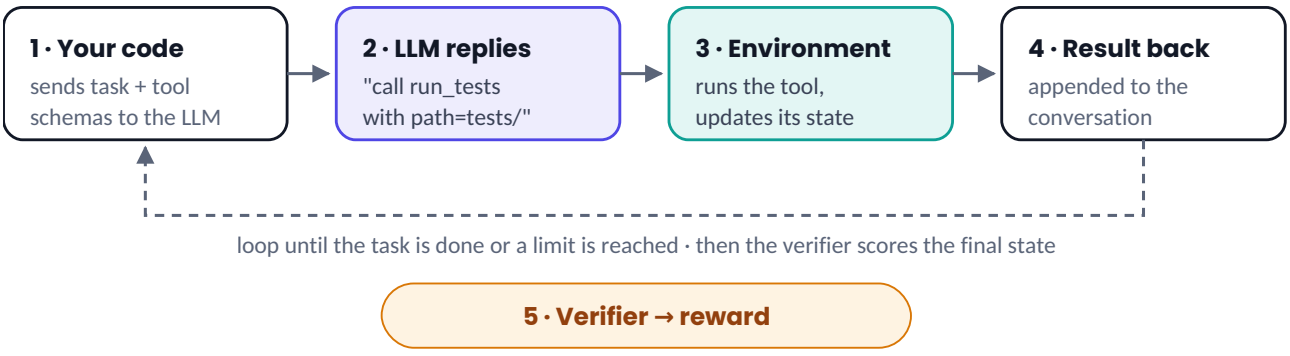
03 From Classic RL to LLM Agents

Classic RL environments are games and physics simulations. LLM agents use the same loop, but the observations are text and the actions are tool calls.

CONCEPT	CLASSIC RL: BALANCING A POLE (CARTPOLE)	LLM AGENT: FIXING A BUG IN A REPO
Observation	Four numbers: cart position, velocity, pole angle, angular velocity	Text: the task description, file contents, test output
Action	Push left or push right	A tool call, e.g. `read_file`, `edit_file`, `run_tests`
Reward	+1 for every step the pole stays up	Score from a verifier, e.g. fraction of tests passing
Episode ends	Pole falls, or 500 steps pass	Agent says it's done, or hits a step / time limit
Agent	A small neural network trained from scratch	A large language model, prompted (and optionally fine-tuned)

How an LLM "takes an action": tool calling

Modern LLM APIs let you describe tools as JSON schemas. The model replies either with text or with a structured request to call one of your tools; your code runs the tool and sends the result back. That back-and-forth is the agent loop.



Every provider does this a little differently

OpenAI, Anthropic, Google and open-source model servers all support tool calling, but message formats differ: how tool calls are represented, how results are sent back, how multiple calls in one turn work. Read at least two providers' docs (links in section 06) so the differences don't surprise you.

04 Hands-On: Your First RL Loop

Every RL setup, from a toy game to a production system, has the same six moving parts.

01
Task
What the agent must achieve, and what counts as success

02
Tools
The actions available, each with a name, description and arguments

03
State
The world's data, which actions change and observations reveal

04
Reset & step
`reset()` starts fresh; `step(action)` applies one action

05
Verifier
Code that checks the outcome and turns it into a reward

06
Termination
When an episode ends: success, failure, or a step limit

A complete toy example in 25 lines

The agent must guess a secret number using a `guess` tool. Not useful on its own, but it has every part above.

```python
import random

# 02 · Tools: described as a JSON schema the LLM can read
GUESS_TOOL = {"name": "guess", "description": "Guess the secret number",
              "parameters": {"type": "object",
                             "properties": {"n": {"type": "integer"}},
                             "required": ["n"]}}

class GuessEnv:
    def reset(self, seed=None):                    # 04 · fresh start
        self.secret = random.Random(seed).randint(1, 100)   # 03 · state
        self.steps, self.solved = 0, False
        return "Find a number from 1 to 100 using the guess tool."  # 01 · task

    def step(self, n):                             # 04 · one action
        self.steps += 1
        if n == self.secret:
            self.solved = True
            return "Correct!", True
        hint = "higher" if n < self.secret else "lower"
        return f"Try {hint}.", self.steps >= 10    # 06 · termination

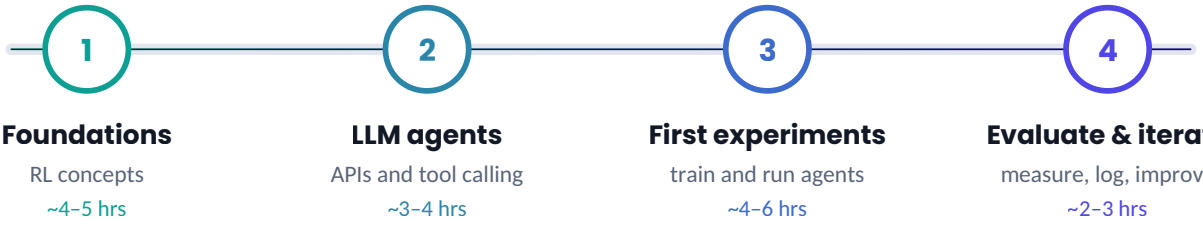
    def verify(self):                              # 05 · verifier
        return 1.0 - 0.05 * self.steps if self.solved else 0.0
```

Try it
Wire this to any LLM with tool calling: send the task and `GUESS_TOOL`, pass each tool call's `n` into `step()`, return the text, and score with `verify()` at the end.

Think about it
What happens if the model sends `n` as a string, or no `n` at all? Real systems have to decide. That's part of the craft.

05 Learning Path

Four stages, roughly 12 to 18 hours in total. Teams can split stages between members and teach each other.



STAGE	DO THIS	YOU'RE READY WHEN YOU CAN…
1 · Foundations	Hugging Face Deep RL Course, Unit 1. Skim Spinning Up's "Key Concepts". Optional: David Silver's Lecture 1.	Explain state, action, reward, policy and return to a teammate
2 · LLM agents	Read the tool-calling guide for two providers. Read "Building Effective Agents". Write a 30-line loop where an LLM calls a calculator tool.	Get an LLM to call your function and use the result
3 · First experiments	Gymnasium's getting-started guide and custom environment tutorial. Train a small agent with Stable-Baselines3. Connect the toy example from section 04 to your stage-2 LLM loop.	Run a full episode end to end and print a score
4 · Evaluate & iterate	Log every step to a JSONL file. Run 10 episodes and look at where the agent fails. Read about reward design. Browse how a benchmark like SWE-bench measures agents.	Explain why your agent scored what it did, from the logs alone

Short on time?

Do stage 2 and stage 3 first. A working understanding of the loop and of tool calling goes a long way; deeper RL theory helps, but it isn't required.

06 Resource Library

All free, and links are clickable in this PDF. Don't try to read everything: pick one resource per category, finish it, and spend the rest of your time building.

RL FOUNDATIONS

Hugging Face Deep RL Course
CODE · BEGINNER

Hands-on course from first principles to modern methods, with notebooks
`huggingface.co/learn/deep-rl-course`

OpenAI Spinning Up
READ · BEGINNER

Concise introduction to deep RL; "Key Concepts" is the best one-hour primer
`spinningup.openai.com`

David Silver's RL Course (UCL)
WATCH · INTERMEDIATE

The classic lecture series; Lectures 1 and 2 cover the core ideas
`davidsilver.uk/teaching`

Sutton & Barto, Reinforcement Learning: An Introduction
READ · DEEP DIVE

The standard textbook, free online; chapters 1 and 3 are enough to start
`incompleteideas.net/book/the-book-2nd.html`

Andrej Karpathy, Deep RL: Pong from Pixels
READ · INTERMEDIATE

A famous, readable walkthrough of policy gradients with code
`karpathy.github.io/2016/05/31/rl`

Hugging Face, Illustrating RLHF
READ · BEGINNER

How reinforcement learning from human feedback is used to train language models
`huggingface.co/blog/rlhf`

LLM AGENTS & TOOL CALLING

Building Effective Agents (Anthropic)
READ · BEGINNER

Practical patterns for agent loops and tool use, provider-agnostic
`anthropic.com/engineering/building-effective-agents`

OpenAI function calling guide
DOCS · BEGINNER

How to define tools and handle tool calls with OpenAI-style APIs
`platform.openai.com/docs/guides/function-calling`

Anthropic tool use guide
DOCS · BEGINNER

The same concept in Anthropic's Messages API format
`docs.claude.com · tool use overview`

Model Context Protocol (MCP)
DOCS · INTERMEDIATE

An open standard for connecting models to tools and data
`modelcontextprotocol.io`

06 Resource Library continued

FRAMEWORKS & LIBRARIES

Gymnasium
CODE BEGINNER

The standard `reset()` / `step()` API; see its "create a custom environment" tutorial
`gymnasium.farama.org`

Stable-Baselines3
CODE BEGINNER

Reliable implementations of standard RL algorithms such as PPO and DQN
`stable-baselines3.readthedocs.io`

Hugging Face TRL
CODE INTERMEDIATE

Library for training language models with RL and preference methods
`huggingface.co/docs/trl`

OpenEnv (Meta PyTorch & Hugging Face)
CODE INTERMEDIATE

Gymnasium-style interfaces for agentic RL, packaged as containers
`github.com/meta-pytorch/OpenEnv`

Verifiers (Prime Intellect)
CODE INTERMEDIATE

Library for training and evaluating LLMs with RL, with a community hub of examples
`github.com/PrimeIntellect-ai/verifiers`

HUD SDK
CODE INTERMEDIATE

Toolkit for running agent evaluations and RL on real software
`github.com/hud-evals/hud-python`

REWARD DESIGN

Lilian Weng, Reward Hacking in Reinforcement Learning
READ INTERMEDIATE

A thorough survey of how reward functions go wrong, with many examples
`lilianweng.github.io`

DeepMind, Specification Gaming
READ BEGINNER

Short, vivid examples of agents satisfying the letter of a goal but not its spirit
`deepmind.google · blog`

AGENT BENCHMARKS TO STUDY

SWE-bench
STUDY INTERMEDIATE

Real GitHub issues as tasks, scored by running the repository's tests
`swebench.com`

Terminal-Bench
STUDY INTERMEDIATE

Agents complete tasks in a real terminal; each task ships with its own tests
`tbench.ai`

Prime Intellect Environments Hub
BROWSE ALL LEVELS

Community-built tasks and evaluations to browse for ideas
`app.primeintellect.ai/dashboard/environments`

07 Warm-up Projects

Three small builds that exercise the skills you'll want on day one. Do them in order; each takes an evening.

Project A · Grid world in Gymnasium

CLASSIC RL · ~2 HOURS

A 5×5 grid where an agent must reach a goal while avoiding walls.

1. Follow Gymnasium's custom environment tutorial to define observation and action spaces.
2. Implement `reset()` and `step()`, with +1 at the goal and a small penalty per step.
3. Run a random agent for 100 episodes and plot the average return.
4. Stretch: train a PPO agent with Stable-Baselines3 and compare.

Project B · A to-do list an LLM can manage

LLM AGENT · ~3 HOURS

A tiny setup where the state is a to-do list and the tools are `add`, `complete` and `list`.

1. Write the tool schemas and a class that holds the list.
2. Give the agent a task such as "add three items, then mark the second one done."
3. Write a verifier that checks the final list and returns a score between 0 and 1.
4. Run it with two different LLMs. Note anything that broke, and why.

Project C · Explore a real framework

FRAMEWORKS · ~3 HOURS

Get hands-on with one of the frameworks from section 06.

1. Install OpenEnv, Verifiers or the HUD SDK and run one of its examples.
2. Read the example's code and find its task, tools, state, verifier and termination.
3. Port your Project B to the framework.
4. Log every step to a JSONL file and read back one full episode.

08 Get Ready Checklist

Tick these off before the event so your time goes into building, not setup.

Setup

- [ ] **Python 3.11+** with `pip` or `uv`, and a virtual environment you know how to create
- [ ] **Git and GitHub** account, with a repo your team can push to
- [ ] **Code editor** set up, with an AI coding assistant if you use one
- [ ] **Docker** installed and able to run `docker run hello-world`
- [ ] **An LLM API key** from any provider, tested with one tool call
- [ ] **One framework** installed and an example running

TOOLS WORTH INSTALLING

Docker: Get Started — CODE, BEGINNER
Package your environment so it runs the same on any machine
`docs.docker.com/get-started`

Pydantic — CODE, BEGINNER
Define and validate data schemas in Python, handy for tool arguments
`docs.pydantic.dev`

LiteLLM — CODE, BEGINNER
One Python interface for calling many LLM providers
`docs.litellm.ai`

JSON Lines — READ, BEGINNER
The simple one-record-per-line format most agent logs use
`jsonlines.org`

See you at the hackathon.

Come curious, come prepared, and bring an idea for a world worth building.